

MEDIFLOW

Documentación de Implementación

TICKET #1 Inicializar el Backend con Spring Boot

Proyecto	MediFlow Backend (G10-MediFlow-Team11)
Stack	Spring Boot 4.1.1 · Java 21 · Maven · PostgreSQL
Paquete base	com.mediflow.backend
Fecha de implementación	19 de septiembre de 2026
Estado	Completado (estructura base + configuración externa)

Este documento describe la implementación real del Ticket #1, adaptada a las decisiones técnicas tomadas durante el desarrollo (versiones, dependencias y configuración final).

1. Objetivo del Ticket

Crear la estructura base del microservicio backend con Spring Boot, Java 21 y Maven, incluyendo las dependencias mínimas necesarias, configuración externa para PostgreSQL (sin credenciales hardcodeadas) y verificación de que el proyecto compila correctamente en local.

2. Decisiones Técnicas Tomadas

Aspecto	Decisión final
Build tool	Maven (con Maven Wrapper)
Spring Boot	4.1.1
Java	21
GroupId / ArtifactId	com.mediflow / backend
Paquete base	com.mediflow.backend
Lombok	Sí (incluido + annotation processor configurado)
Formato de configuración	application.yml
Soporte fechas JSON	jackson-datatype-jsr310 + write-dates-as-timestamps: false
Actuator	Exposición de health e info

3. Dependencias Incluidas (pom.xml)

Se utilizó Spring Initializr (vía IntelliJ) y se ajustó el `pom.xml` con las siguientes dependencias:

Dependencia	Propósito
spring-boot-starter-web	API REST / servidor embebido
spring-boot-starter-data-jpa	Persistencia ORM (Hibernate)
postgresql (runtime)	Driver JDBC para PostgreSQL
spring-boot-starter-validation	Validación de DTOs (@Valid, constraints)
spring-boot-starter-actuator	Health checks y endpoints de monitoreo
lombok (optional)	Reducción de boilerplate (getters, setters, etc.)
jackson-datatype-jsr310	Serialización correcta de fechas Java 8+
spring-boot-starter-test	JUnit 5, Mockito y soporte de tests Spring

Nota: Se configuró explícitamente el *annotation processor* de Lombok en el *plugin* `maven-compiler-plugin` para ambos *goals* (`compile` y `testCompile`).

4. Configuración de la Aplicación

4.1 application.yml

Se eliminó el `application.properties` generado y se creó `application.yml` con configuración externalizada mediante variables de entorno y valores por defecto solo para desarrollo local:

```
# Configuración del servidor web
server:
  port: ${SERVER_PORT:8080}

spring:
  application:
    name: mediflow-backend

  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:mediflow}
    username: ${DB_USER:mediflow_user}
    password: ${DB_PASSWORD:changeme}
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
    open-in-view: false

  jackson:
    serialization:
      write-dates-as-timestamps: false

management:
```

```
endpoints:
  web:
    exposure:
      include: health,info
```

Puntos clave de la configuración:

- Ninguna credencial está hardcodeada: se usan placeholders `${VAR:default}`.
- `ddl-auto`: `update` permite sincronizar entidades con el esquema en desarrollo (aceptable para el MVP).
- `open-in-view`: `false` evita el anti-patrón de sesión de Hibernate abierta durante el renderizado.
- `write-dates-as-timestamps`: `false` fuerza el formato ISO-8601 en las fechas JSON.
- Actuator expone solo `/actuator/health` y `/actuator/info`.

4.2 Archivo `.env.example`

Se creó en la raíz de `backend/` el archivo `.env.example` para documentar las variables necesarias:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mediflow
DB_USER=mediflow_user
DB_PASSWORD=
SERVER_PORT=8080
```

Se actualizó el `.gitignore` del repositorio para excluir archivos `.env` reales y preservar solo el `.env.example`.

5. Código Base Generado

5.1 Clase principal

`com.mediflow.backend.BackendApplication`

```
@SpringBootApplication
public class BackendApplication {
    public static void main(String[] args) {
        SpringApplication.run(BackendApplication.class, args);
    }
}
```

5.2 Test de contexto

Se mantiene el test generado por Spring Initializr. Este test fallará hasta que exista una instancia de PostgreSQL disponible (dependencia del Ticket #14), lo cual es el comportamiento esperado en esta etapa.

```
@SpringBootTest
class BackendApplicationTests {
    @Test
    void contextLoads() {
    }
}
```

6. Verificación Realizada

Se verificó que el proyecto compila correctamente mediante Maven Wrapper:

Windows (PowerShell):

```
.\mvnw.cmd clean compile
```

Resultado esperado: **BUILD SUCCESS**. El arranque completo de la aplicación fallará al intentar conectar con PostgreSQL (aún no disponible), lo cual es correcto y se resolverá en el Ticket #14.

7. Criterios de Aceptación Cumplidos

#	Criterio	Estado
1	Proyecto Spring Boot creado con Java 21 y Maven	Cumplido
2	Dependencias mínimas configuradas (Web, JPA, PostgreSQL, Validation, Actuator, Lombok)	Cumplido
3	Configuración de datasource externalizada (sin hardcode)	Cumplido
4	application.yml con valores por defecto seguros para desarrollo local	Cumplido
5	Archivo .env.example documentando variables requeridas	Cumplido
6	.gitignore actualizado para proteger archivos .env	Cumplido
7	Proyecto compila sin errores (mvn clean compile)	Cumplido
8	Estructura lista para continuar con tickets posteriores	Cumplido

8. Notas y Observaciones

- El warning del IDE sobre `write-dates-as-timestamps` (Cannot resolve enum constant en `tools.jackson.databind.SerializationFeature`) es un falso positivo del plugin de Spring/YAML de IntelliJ. La propiedad es válida y Spring Boot la resuelve correctamente en runtime.
- El arranque completo de la aplicación depende de la existencia de PostgreSQL (Ticket #14). Hasta entonces, el `test contextLoads` y el arranque fallarán por conexión a BD, comportamiento esperado.
- Se utilizó Spring Boot 4.1.1 (versión generada por el Initializr en el momento de la creación). La estructura es compatible con las prácticas recomendadas para microservicios Spring Boot modernos.
- Lombok fue incluido y el annotation processor quedó correctamente configurado en el `pom.xml` para evitar problemas de compilación.

Documento generado a partir de la implementación real del Ticket #1.

Listo para adjuntar como evidencia en el Pull Request o como parte de la entrega del hackathon.